

EXPERIMENT-03

C Programming - Debugging Tools

1. Fundamental of Errors and Debugging

Key Definitions

- **Error:** A mistake or blunder made by the programmer in the source code.
- **Bug:** A flaw or fault in a computer program that cause it to produce an incorrect or unexpected result, or to behave in unintended ways.
- **Debugging:** The systematic process of finding, analysing, and removing bugs or defects in a computer program.

2. Classification of Errors in C

Error Type	Description	Primary Cause	Example
Syntax Errors	Code violates the grammatical rules of the C language.	Missing semicolons, unmatched braces, typos in keywords.	int a = 10 (missing
Compilation/Type Error	Operation performed on incompatible types or missing declarations.	Assigning float string to int pointer, using undeclared variable.	Int *p = "hello";
Linker Errors	The compiler generates object files, but the linker cannot combine them.	Function declared but not defined, missing libraries.	Undefined reference to 'main'
Runtime Errors	The program crashes or terminates abnormally during execution.	Division by zero, accessing invalid memory addresses.	Int x = 10 / 0;
Logical Errors	Code runs to completion without crashing, but output is incorrect.	Wrong mathematical formula, incorrect loop condition.	for(I=0; I<5; I++) instead of <=5

Why Does a Program Compile Successfully but Produce Incorrect Output?

A compiler's job is to verify that your code obeys the **syntax** and **type rules** of the C language, and to convert that code into machine instructions. The compiler **cannot understand intent**.

- If you write `sum =a - b;` instead of `sum = a + b;` the statement is completely valid C syntax. Therefore, compilation succeeds, but the resulting logic produces an erroneous output at execution time.

Case 1: Syntax & Compilation Errors

When the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column, and error description

```
#include <stdio.h>
Int main()
{
Int x = 10;
Printf("the value of x is %d" , x)
Return 0;
}
```

Compilation Command & Output

```
$ cc sample1.c
sample1.c:5:5: error: expected ',' or ';' before 'printf'
```

Breakdown of Error Message

- **Filename:** sample1.c
- **Line & Column:** Line 5, Column 5
- **Type of Message:** error
- **Description:** Expected , or ; before printf statement.



This is an example of syntax and compilation error
Where we didn't place an semicolon after the

Int a = 10 and the second pic depicts the
compilation error where the compiler detected the
bug in the code.

In the above to pics we have debugged the code
and it is running how it is supposed to.

```

1  #include <stdio.h>
2  int main()
3  {
4      int a = 10
5      printf("%d\n", a);
6      return 0;
7  }

```

Case 2: Compiler Warnings (-Wall, -Wextra)

A **warning** indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do **not** stop compilation, but ignoring them frequently leads to bugs at runtime.

```

1  #include <stdio.h>
2  int main()
3  {
4      int a;
5      printf("Hello\n");
6      return 0;
7  }
8

```

```

start pause continue step over step into step out help
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwy64npv-glibc-2.40-66/lib/libthread_db.so.1".
Hello
[Inferior 1 (process 6647) exited normally]
(gdb)

```

```

#include <stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: entering iteration i = %d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
    }
    printf("Final sum updated to = %d\n", sum);
    return 0;
}

```

This code will run if we compile it normally with the command `cc sample2.c` but if we use the command `cc -Wall -Wextra sample2.c` it will show an unused variable.

Case-3: printf-Based Debugging (Logical bug)

Before using complex debugging tools, simple program flow and state inspection can be performed using output statements (`printf`).

```

DEBUG: entering iteration i = 0
DEBUG: sum updated to = 0
DEBUG: entering iteration i = 1
DEBUG: sum updated to = 1
DEBUG: entering iteration i = 2
DEBUG: sum updated to = 3
DEBUG: entering iteration i = 3
DEBUG: sum updated to = 6
DEBUG: entering iteration i = 4
DEBUG: sum updated to = 10
Final sum updated to = 10

```

Case 4: Debugging a Logical Error using GDB

NOW WE WILL USE GDB TO DEBUG THE CODE.

```
1  #include <stdio.h>
2  int main()
3  {
4      int i, sum = 0;
5      for(i = 1; i < 5; i++)
6      {
7          sum = sum + i;
8      }
9      printf("sum = %d\n", sum);
10     return 0;
11 }
```

```
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at main.c:7
7          sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
sum = 10
[Inferior 1 (process 5014) exited normally]
(gdb)
```

Case 5: Debugging Functions (next vs. step)

In this we will debug using two steps

- 1) next
- 2) step

By using next to debug

```
#include <stdio.h>
int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}
int main()
{
    int result, n = 5;
    result = square(n);
    printf("square = %d\n", result);
    return 0;
}
```

```
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 2, main () at main.c:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) step
square (x=5) at main.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb)
```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

```
#include <stdio.h>
int divide(int a, int b)
{
    return a / b;
}
int calculate(int x)
{
    return divide(x, 0);
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

Breakpoint 2, `calculate (x=10)` at `main.c:8`

```
8         return divide(x, 0);
```

(gdb) continue

Continuing.

Program received signal SIGFPE, Arithmetic exception.

0x000055555555147 in `divide (a=10, b=0)` at `main.c:4`

```
4         return a / b;
```

(gdb) backtrace

#0 0x000055555555147 in `divide (a=10, b=0)` at `main.c:4`

#1 0x000055555555166 in `calculate (x=10)` at `main.c:8`

#2 0x00005555555517a in `main ()` at `main.c:12`

(gdb)

Case 7: Debugging Array Out-of-Bounds Errors using GDB

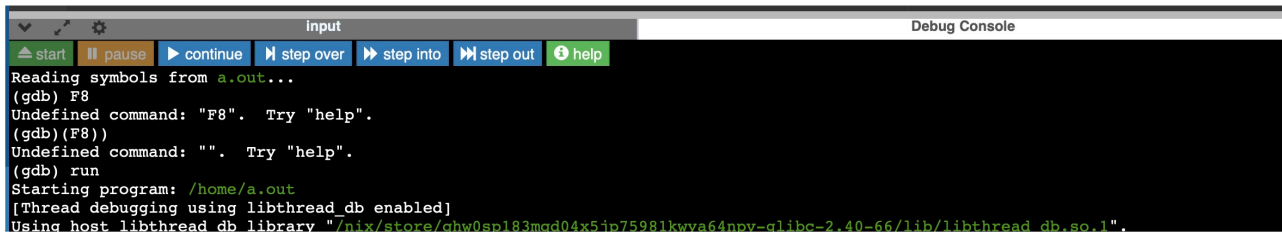
```
#include <stdio.h>
int main()
{
    int a[5];
    int i;
    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]);
    }
    return 0;
}
```

```
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:8
8      scanf("%d", &a[i]);
(gdb) print &a[0]
$1 = (int *) 0x7fffffffefa60
(gdb) print &a[4]
$2 = (int *) 0x7fffffffefa70
(gdb) print &a[5]
$3 = (int *) 0x7fffffffefa74
(gdb)
```

Case 8: Segmentation Faults (SIGSEGV) using GDB

```
//Case-8 Segmentation Faults (SIGSEGV) using GDB
#include <stdio.h>
int main()
{
    int *p = NULL; //Pointer points to address 0x0
    *p = 10;       //Attempting to write to address 0x0 causes Segmentation Fault
    printf("%d\n", *p);
    return 0;
}
```

The screenshot shows a GDB interface with a toolbar at the top containing buttons for start, pause, continue, step over, step into, step out, and help. Below the toolbar, the text "Reading symbols from a.out..." is displayed. The user has entered the command "(gdb) F8", which resulted in the message "Undefined command: 'F8'. Try 'help'.". The user then entered "(gdb) run", and the program started. The output shows "Starting program: /home/a.out" and "[Thread debugging using libthread_db enabled]". The final line of output is "Using host libthread_db library '/nix/store/gbw0sp183mqd04x5jn75981kwva64npv-glibc-2.40-66/lib/libthread_db.so.1'".

```
Reading symbols from a.out...
(gdb) F8
Undefined command: "F8". Try "help".
(gdb)(F8)
Undefined command: ". Try "help".
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/gbw0sp183mqd04x5jn75981kwva64npv-glibc-2.40-66/lib/libthread_db.so.1".
```

```
Program received signal SIGSEGV, Segmentation fault.
0x00005555555514d in main () at main.c:6
6      *p = 10;          //Attempting to write to address 0x0 causes Segmentation Fault
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
Cannot access memory at address 0x0
(gdb)
```

In this experiment, we learned how to debug source code using method such as GDB and printf.